CS780 Capstone Project: OBELIX: The Warehouse Robot

Deepak Chaurasia
220330
deepakc22@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This project addresses the OBELIX robot box-pushing task using deep reinforcement learning under sparse rewards and partial observability.I have implemented a Dueling Double Deep Q-Network (D3QN) enhanced with Prioritized Experience Replay (PER) and n-step returns for improved sample efficiency. To tackle reward sparsity, we designed a three-component Potential-Based Reward Shaping (PBRS) scheme incorporating bot-to-box distance, heading alignment, and box-to-boundary proximity gradients.I further integrated Random Network Distillation (RND) and count-based exploration bonuses to encourage systematic state-space coverage. Observation stacking provided temporal context for the partially observable environment, while curriculum learning across four difficulty phases enabled progressive skill acquisition. Forward-biased exploration and stuck-aware action selection improved navigation efficiency in wall-constrained arenas.I also tried some policy based algorithms like PPO but that doesn't worked out here,one reason could be like it require large number of episodes for training and OBELIX was descrete action but policy based like PPO work well with continous action space.

1 Competition Result

Codabench Username: d_220330
Final leaderboard rank (Test Phase): 22
Leaderboard rank – Level 1 (static): 28
Leaderboard rank – Level 2 (blinking): 11
Leaderboard rank – Level 3 (moving + blinking): 9

Number of submissions made (Development(Level 1 , Level 2, Level 3) and Test Test):

Submission Type	Level 1	Level 2	Level 3	Test	Total
Development	6	5	3	3	17

Table 1: Number of Valid Submissions (Development and Test)

2 Introduction and Problem Description

- **Motivation:**

  Autonomous object manipulation in robotics requires coordinated perception, navigation, and interaction under uncertainty. In the OBELIX environment, an agent must locate, attach to, and push a box to the boundary using only sparse sensor feedback. The observation space is limited to an 18-dimensional binary vector, lacking rich spatial or visual information. This makes the problem highly challenging and motivates the use of deep reinforcement learning for efficient exploration and control.

- **Core task:**

  The OBELIX task consists of a sequential, multi-phase objective:

  Find: The robot must explore the arena to locate the grey box using 16 sonar sensors (8 positions $\times$ 2 ranges: near and far) and 1 forward-facing infrared (IR) sensor. All sensors return binary values (0 or 1), providing only coarse proximity information.

Attach: Once the IR sensor detects the box at close range, the robot transitions to a "pushing" state (enable_push = True), enabling physical interaction with the box.

Push: The robot must push the attached box toward the nearest arena boundary. It moves in the direction the robot pushes, and successful delivery to the boundary earns a +2000 bonus reward.

Unwedge: If the box gets stuck against a wall obstacle, the robot must manoeuvre around the obstacle and reposition to continue pushing.

The robot operates in a 500×500 pixel arena with 5 discrete actions: turn left 45°, turn left 22.5°, move forward, turn right 22.5°, and turn right 45°. Each action moves the robot by 5 pixels when moving forward. Episodes terminate after a maximum of 2000 steps, with a per-step reward composed of sensor activation bonuses (sonar: +1 to +3, IR: +5) offset by a penalty of 1 for each inactive sensor and 200 for being stuck. This reward structure is inherently sparse: the dominant signal is the +2000 success bonus, while the rewards per step are small and largely negative (typically 18 per step when no sensors detect anything), resulting in cumulative rewards.

- **Environment:**

  The observation space is represented as an 18-dimensional binary vector

Index	Sensor	Description
0–3	Left sonar (×4)	2 positions × (far, near) range on the left side
4–11	Forward sonar (×8)	4 positions × (far, near) range facing forward
12–15	Right sonar (×4)	2 positions × (far, near) range on the right side
16	IR sensor	Forward-facing infrared sensor; detects box at close range
17	Stuck flag	Set to 1 if the robot is stuck against a wall or obstacle

Table 2: Observation Space Representation (18-bit Sensor Vector)

Each sonar sensor operates with two detection ranges: a **far range** (scaled by 30) and a **near range** (scaled by 18), each covering a 20° field of view. A sensor outputs 1 if any object (box, wall, or boundary) is detected within its cone, and 0 otherwise.

The infrared (IR) sensor has a shorter detection range (scaled by 4) and plays a crucial role in identifying close proximity to the box, which is essential for successful attachment.

- **Difficulty levels :**

  The environment defines three difficulty levels that increase the task complexity:

Difficulty	Box Behaviour	Description
0 (Static)	Stationary	The box remains fixed at its spawn location. The agent only needs to locate and push it to the boundary.
2 (Blinking)	Periodic visibility toggling	The box alternates between visible (30–60 steps) and invisible (10–30 steps) periods. The agent must handle intermittent sensor signals and rely on past observations to estimate the box's position.
3 (Moving + Blinking)	Random motion + blinking	The box moves along a random trajectory at a configurable speed while also blinking. The agent must track and intercept a moving, intermittently visible target.

Table 3: Difficulty Levels in the OBELIX Environment

The environment supports an optional wall obstacle mode, where rectangular wall segments are placed randomly in the arena. These walls occlude sensors, block forward movement (triggering the stuck flag).

- **POMDP setting :**

  The OBELIX environment constitutes a Partially Observable Markov Decision Process (POMDP) for several reasons:

  Binary, low-dimensional observations: The 18-bit sensor vector provides only a coarse, binary snapshot of the surroundings. The agent cannot determine absolute positions, distances, or velocities it sees only whether something is within each sensor cone.

  Limited sensor range: Sonar range is bounded, so the box is undetectable when far away. The agent must explore blindly until the box enters sensor range.

  No positional information: The agent has no access to its own coordinates, heading, or the box's position. Navigation must be learned purely from sequential sensor patterns.

  Blinking and moving targets: At higher difficulties, the box periodically disappears from perception and moves unpredictably, requiring the agent to maintain temporal context.

  Wall occlusion: Walls block sensor cones, creating dead zones where the box is physically present but undetectable.

- **High-level goal:** Our objective is to maximise the mean cumulative reward across all difficulty levels (static, blinking, and moving+blinking).

3 Approach and Model Evolution

3.1 Baseline and Initial Attempts

3.1.1 Phase 1: Vanilla DQN (Mean Reward: ∼130,000)

Our first approach used a standard Deep Q-Network (DQN) with a simple 3-layer MLP (18→64→64→5) operating on raw 18-dimensional binary observations. The agent used uniform ϵ-greedy exploration (ϵ decaying from 1.0 to 0.05) and a basic experience replay buffer.

Why it failed:

- The 18-bit observation provides no positional information – the agent could not distinguish between states that "look" the same from different positions in the arena.
- Uniform random exploration caused the agent to spin in place or wander aimlessly, rarely discovering the box within the 2000-step budget.
- The per-step reward is dominated by the -1 penalty for each inactive sensor (typically -18/step), providing almost no gradient toward the box.

3.1.2 Phase 2: Double DQN (Mean Reward: ∼100,000)

We upgraded to Double DQN (DDQN), which uses the policy network to select actions and the target network to evaluate them, addressing the Q-value overestimation problem inherent in standard DQN.

Improvement: ∼23% better mean reward than DQN, indicating that overestimation was partially responsible for poor action selection. However, the fundamental exploration and partial observability challenges remained unsolved.

3.2 Evolution Across Phases

3.2.1 Phase 3: D3QN – Dueling Double Deep Q-Network, Best Performing

We adopted the Dueling architecture (Wang et al., 2016), which decomposes $Q(s, a)$ into a state-value stream $V(s)$ and an advantage stream $A(s, a)$:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a') \right)$$

Why this helped: In OBELIX, many states have similar value (e.g., the agent is far from the box with no sensor activations). The dueling architecture lets the network learn "this state is bad" (low V) without needing to evaluate every action separately, improving generalisation across the large, mostly-uniform state space.

3.2.2 Phase 4: Prioritised Experience Replay (PER) + N-step Returns

We added two standard DQN improvements:

- **PER** (Schaul et al., 2016): Samples transitions proportional to their TD-error magnitude, ensuring that surprising or informative transitions are replayed more frequently. This is critical in OBELIX where the vast majority of transitions are uninformative (wandering with no sensor activations), while the rare box-discovery transitions carry the most learning signal.
- **N-step returns** (n=3): Instead of the 1-step TD target $r+\gamma Q(s')$, we use the 3-step target $r_1+\gamma r_2+\gamma^2 r_3+\gamma^3 Q(s_3)$. This propagates the sparse +2000 success reward backwards through 3 steps per update, tripling the effective reward propagation speed.

3.2.3 Phase 5: Observation Stacking (6 frames → 108-dim)

To address partial observability, we adopted frame stacking from Atari DQN (Mnih et al., 2015), concatenating the last 6 observations into a single 108-dimensional input vector:

$$input_t = [o_{t-5}, o_{t-4}, o_{t-3}, o_{t-2}, o_{t-1}, o_t]$$

Why this helped: A single 18-bit reading tells the agent what it sees *now*, but not where it came from or how sensors changed over time. With stacking, the agent can learn temporal patterns like "sensors detected something on the left 2 steps ago → now nothing → the box is behind me, I should turn around."

3.2.4 Phase 6: Three-Component Reward Shaping (PBRS),Most Impactful

We designed a Potential-Based Reward Shaping (PBRS) scheme with three components, each using the $\gamma\Phi(s') - \Phi(s)$ formulation that is mathematically guaranteed to preserve optimal policy (Ng et al., 1999):

Component	Potential Function $\Phi(s)$	When Active	Scale	Purpose
Bot→Box distance	$-dist(bot, box)/max_dist$	Always	1.0×	Guide agent toward box
Heading similarity	$\cos(facing, toward_box)$	Before attachment	0.5×	Reward facing the box
Box→Boundary dist	$-dist(box, boundary)/max_dist$	After attachment	2.0×	Guide pushing direction

Table 4: Three-component potential-based reward shaping used in OBELIX.

Why three components, not just distance:

- Distance-only shaping: The agent learns to approach the box but may circle it without facing it – never attaches.
- Adding heading: The agent turns toward the box, achieving faster attachment.
- Adding box→boundary: Without this, the pushing phase has zero gradient – the agent pushes randomly. The boundary potential creates a directional signal toward the nearest wall.

3.2.5 Phase 7: Forward-Biased Exploration

Standard ϵ-greedy selects random actions uniformly, but in OBELIX the forward action (FW) is disproportionately useful – the agent needs to traverse the arena, not spin in place. We implemented non-uniform random action selection:

Action	Probability
L45	5%
L22	15%
FW	**60%**
R22	15%
R45	5%

Table 5: Non-uniform exploratory action probabilities.

Additionally, we implemented **stuck-aware exploration**: when the stuck flag (obs[17]) is active, the forward action is masked and only turning actions are available, preventing the agent from wasting steps bumping into walls.

3.2.6 Phase 8: Curriculum Learning (4 Phases)

Training on the hardest setting(with wall) from the start caused catastrophic failure – the agent never learned to find the box when walls blocked its path. We designed a 4-phase curriculum:

Phase	Episodes	Walls	Difficulties	Purpose
P1: Warm-up (No wall)	1–300	No	Static (0)	Learn basic find→push
P2: Wall intro	301–1000	Yes	Static (0)	Learn wall navigation
P3: Blink intro	1001–1500	Yes	Static (0) + Blinking (2)	Handle intermittent visibility
P4: Full	1501–2000	Yes	All (0, 2, 3)	Generalise across all difficulties

Table 6: Curriculum learning schedule used in training.

3.3 Experimental Approaches (Tried and Reverted)

3.3.1 Action Masking Near Walls – REVERTED

We implemented wall detection using forward sonar sensors: when sensors 4–11 (forward-facing) indicated an obstacle, the forward action was masked, forcing the agent to turn.

Why we reverted: Hard masking removed the forward action from the Q-value computation entirely, causing gradient instability. The agent could not learn that "in this state, forward is bad" because it never saw the Q-value for forward in wall-adjacent states.

3.3.2 Exploration Bonuses (RND + Count-Based) – REMOVED

We implemented two intrinsic motivation methods(from NeurIPS Paper):

1. **Random Network Distillation (RND)** (Burda et al., 2019): A fixed random network and a trainable predictor; the prediction error serves as a novelty bonus for unvisited states.

2. **Count-Based Exploration** (Tang et al., 2017): Since OBELIX observations are binary, we directly hash state vectors and track visit counts, providing a $\beta/\sqrt{N}$ bonus for rarely-visited states.

Why we removed them: While theoretically sound, these bonus required careful tuning of the bonus scale (β) relative to the PBRS shaping. The PBRS already provides strong directional gradients, and adding exploration bonuses on top created conflicting reward signals – the agent would sometimes prioritise visiting novel states over approaching the box.

3.4 Proximal Policy Optimization (PPO) – Alternative Architecture

As a parallel approach, we implemented(PPO) with a shared-backbone Actor-Critic architecture. Unlike D3QN (off-policy, value-based), PPO is an on-policy policy-gradient method that directly optimises the policy.But it was not working well because generally for on-policy algorithm number episodes requires is more but due to compute constraint I cannot run beyond 2000 episodes.

3.5 Final Model: Detailed Description

3.5.1 Algorithm: Dueling Double Deep Q-Network (D3QN)

Our best-performing model is D3QN with the following enhancements: PER, n-step returns, observation stacking, PBRS, forward-biased exploration, and curriculum learning.

3.5.2 State Representation

We use the raw 18-bit binary sensor vector without hand-crafted features, relying on observation stacking (6 frames) to provide temporal context. The stacked 108-dimensional input preserves all sensor information while enabling the network to learn temporal patterns.

3.5.3 Exploration Strategy

ϵ-greedy with:

- Linear decay from $\epsilon = 1.0$ to $\epsilon = 0.05$ over 200,000 steps
- Forward-biased random actions (60% FW, 15% L22/R22, 5% L45/R45)
- Stuck-aware masking (no forward when stuck flag is active)

3.5.4 Network Architecture

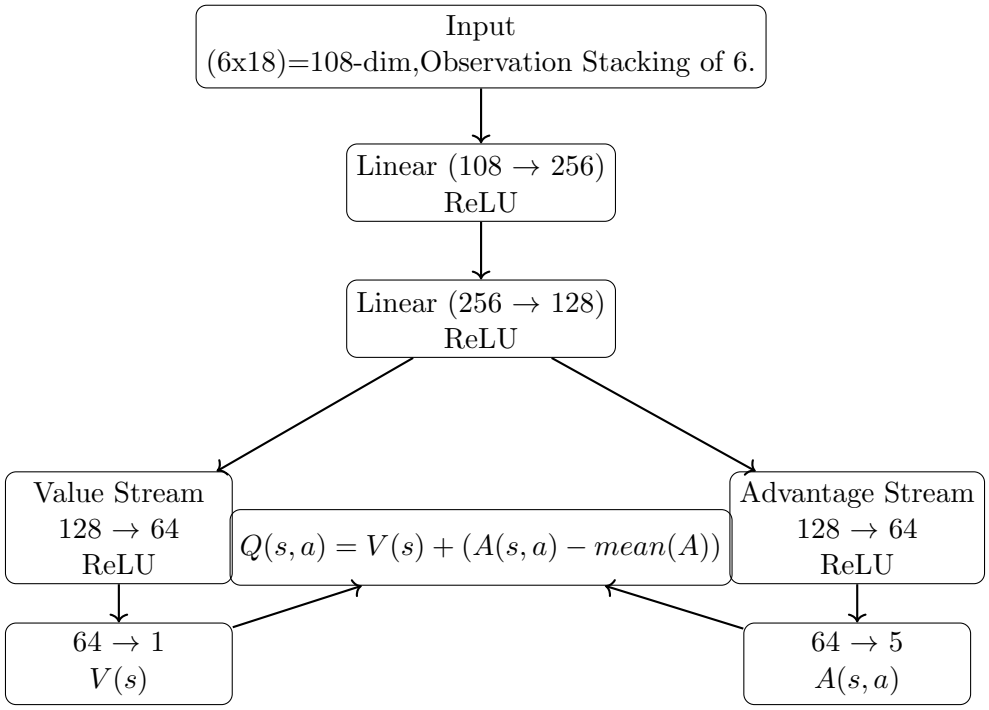

Figure 1: Dueling Double DQN Network Architecture : Total Parameter = 47,000

3.5.5 Reward Shaping

Three-component PBRS described in Section 3.2, Phase 6. Shaping scale factor = 10.0. This provides dense per-step gradients while maintaining the theoretical guarantee of preserving the optimal policy.

4 Experiments and Implementation Details

- Training setup: episodes, max steps (2000), hardware (CPU/GPU), time taken = Approx 12 hours
- Hyperparameter table :

Parameter	**Value**	**Rationale**
Learning rate	1×10^{-4}	Stable with PER importance sampling
Discount (γ)	0.99	Long-horizon task (up to 2000 steps)
Batch size	64	Balance between stability and speed
Buffer capacity	50,000	∼25 episodes of experience
PER α	0.6	Standard PER prioritisation
PER β	0.4→1.0	Annealed over 200K steps
N-step	3	Faster reward propagation
Target update	Every 2000 steps	Standard hard update frequency
Observation stack	6 frames	Temporal context window
Shaping scale	10.0	Balanced with env reward magnitude

Table 7: Final hyperparameter configuration used for D3QN training.

5 Results

- Performance of best models across phases

Phase	Phase 1 (L1)	Phase 2 (L2)	Phase 3 (L3)	Test Phase
Best Mean Reward	-1980.70	-1977.40	-1420.32	-1796.41

Table 8: Performance of best models across development and test phases

- **Learning Curves:**

Figure 2: Training learning curve showing episode-wise reward progression

- D3QN-Per with some modification performed best(Explained above why it performed well).

6 Error Analysis and Discussion

- Explained above in each section of experimentations.

7 Conclusion

- **Key achievements :**

  Performance table from DQN (minus 130K) to final D3QN, plus 4 things learned (reward shaping most impactful, essential curriculum, stacking for POMDP, not all enhancements compose).

Model	Approx. Mean Reward	Key Addition
Vanilla DQN	−130,000	Baseline
Double DQN	−100,000	Reduced Q-value overestimation
D3QN + PER + n-step	Improved	Dueling architecture, prioritised replay, faster reward propagation
D3QN + Stacking + PBRS + Curriculum	Best	Temporal context, dense reward gradients, staged training

Table 9: Comparison of Different DQN Variants and Their Performance

1.Reward shaping was the most impactful change transforming a sparse-reward problem into one with dense per-step gradients.

2.Curriculum learning was essential training on the hardest setting from the start caused catastrophic failure.

3.imple observation stacking was sufficient for partial observability — no need for complex recurrent architectures.

4.Not all enhancements compose well NoisyNet, action masking, and exploration bonuses (RND, count-based) degraded performance when combined with PBRS.

- **Most important takeaway :**

  1.The Perception Gap 3 concrete problems (perceptual aliasing, no positional memory, sensor occlusion) + what happens when you fix only this (stacking without PBRS → memory without purpose)

  2.The Reward Gap concrete reward math (36K for failure episodes) + what happens when you fix only this (PBRS without stacking → purpose without the ability to follow it)

- **Possible future directions :**

  1. Recurrent Networks for Longer Memory. Our 6-frame stacking gives the agent memory of only 6 steps. Replacing the feedforward network with an LSTM or GRU (as in DRQN or R2D2) would let the agent remember events from much further back for example, where the box was before it blinked off 30 steps ago. This is especially useful for blinking and moving box difficulties where the agent needs to maintain a belief about the box's location during invisible periods.

  3. Hierarchical RL. The task has a natural structure: Explore → Approach → Push → Unwedge. Instead of one flat policy learning everything, a high-level controller could choose sub-tasks while specialised low-level policies handle each one. This would make learning easier since each sub-policy only needs to solve a simpler problem, and the manager only decides when to switch between them.

  4. Multi-Box and Multi-Agent. Extending to multiple boxes would require the agent to plan which box to push first and find efficient routes. Adding multiple robots introduces coordination challenges — dividing work, avoiding collisions, and sharing partial observations — turning the problem into a decentralised POMDP relevant to warehouse automation.

  5. Learned Reward Shaping (Inverse RL). We hand-designed our 3-component PBRS using domain knowledge. Inverse RL could instead learn the reward function automatically from a few expert demonstrations, potentially discovering better shaping signals we did not think of — like rewarding approach from the side nearest the boundary.

  6. Go-Explore for Hard Exploration. When the box is far away and all sensors read zero, even PBRS provides a weak gradient. Go-Explore (Ecoffet et al., 2021) addresses this by archiving "interesting" states (e.g., first sonar detection) and returning to them in future episodes, focusing training on the skill-intensive phases rather than wasting steps on blind wandering.

7.1 Submissions Links

- **Code Link : https://github.com/Deepakc766/OBELIX-The-Warehouse-Robot**
- **Video Link : https://youtu.be/GU9idDc9qTE?si=FIzfGHkwbcVdsJDI**

LLM Usage Declaration

- Which Large Language Model (LLM) did you use to complete the competition or write the report?

  Mostly Copilot and chat gpt.

- In which part did you use the LLM, and how did it help you?

Part	Used LLM? (Yes/No)	Purpose of Using LLM
Understanding concepts	Yes	Sometimes in summarising paper for new approach
Ideation / brainstorming	No	
Debugging small code snippets	Yes	Sometimes in debugging
Plotting / visualization code	Yes	Mostly for plotting Graphs and making dashboard above
Grammar / writing style	Yes	For rephrasing in report
Report structure / section ideas	No	Report structure were already given
Other (specify)	Yes	For latex code and making above flow chart of network

- Did you refer to any other sources or websites apart from the LLM and lecture slides?
  If yes, please mention clearly (papers, blogs, GitHub repos, textbooks etc.)

  Yes,I have mentioned in reference section.

- Apart from any LLM usage, what was your own contribution?
  Briefly describe your personal understanding, reasoning, implementation effort, analysis, and experimentation.

  1.Reading and searching the ideas and the paper around autonomous robot.

  2.Implementation of code and setting the optimal parameter by researching and from previous experience.

  3.Brainstorming and experimentation with different approaches.

  4.Training the agent and validating and experimenting.

References

https://cdn.aaai.org/AAAI/1991/AAAI91-120.pdf

https://arxiv.org/pdf/1810.12894

https://arxiv.org/pdf/1611.04717

https://arxiv.org/pdf/1707.01495

https://arxiv.org/pdf/2507.04372

https://www.youtube.com/watch?v=A39cjchWnsU

https://github.com/nubot-nudt/SG-D3QN

https://gibberblot.github.io/rl-notes/single-agent/reward-shaping.html

https://arxiv.org/html/2408.10215v1

https://www.youtube.com/watch?v=70kVz9G2Z-o

https://www.youtube.com/watch?v=S6X7Kb7v3II

https://arxiv.org/pdf/2011.02669

https://arxiv.org/pdf/2003.04960

https://medium.com/@heyamit10/reinforcement-learning-with-curriculum-learning-for-complex-tasks-4f306470ed42

https://arxiv.org/abs/2506.06632 and https://arxiv.org/pdf/2108.10392

https://gibberblot.github.io/rl-notes/single-agent/reward-shaping.html

https://medium.com/@jonathan$_hui/rl-dueling-deep-q-network-dueling-dqn-architecture-3f6f6c2b9b8f$

https://medium.com/@jonathan$_hui/rl-prioritized-experience-replay-12a5c13f9b1e$

https://towardsdatascience.com/deep-q-learning-dqn-ii-b6bf911b6b2c

https://www.youtube.com/watch?v=5P7I-xPq8u8t=69s and https://www.youtube.com/watch?v=70kVz9G2Z-o

https://www.youtube.com/watch?v=S6X7Kb7v3II